

```
"""
```

```
fred_fetch.py
```

```
-----
```

Automatically builds the fundamental data table from FRED (Federal Reserve Economic Data - free, <https://fred.stlouisfed.org>), instead of you typing values into the CSV by hand.

Get a free API key: [https://fredapi.stlouisfed.org/docs/api/api\\_key.html](https://fredapi.stlouisfed.org/docs/api/api_key.html)  
(takes ~2 minutes, no cost)

Then either:

```
export FRED_API_KEY=your_key_here
```

or pass --fred-key on the command line to engine.py.

--- Honesty about coverage ---

FRED has excellent, verified series for the US and Euro Area, and solid central-bank-rate series for UK/Japan/Canada. For the rest (AUD, CHF, NZD, MXN, ZAR, TRY, SEK) and for less-standardized metrics (trade balance, retail sales) in general, this module falls back to FRED's series-search API at runtime and takes the best match. That's inherently best-effort -- search ranking isn't perfect. Every value this module writes into the output CSV is visible and editable before you run the analysis, specifically so you can sanity-check or correct anything that looks wrong.

PMI (manufacturing/services) has no reliable free API source (S&P Global/ISM data is paywalled) -- those two columns are left blank for you to fill in manually if you want them included; if left blank they're excluded from that currency's score rather than penalizing it.

```
"""
```

```
import sys
```

```
import time
```

```
import requests
```

```
import pandas as pd
```

```
import numpy as np
```

```
FRED_BASE = "https://api.stlouisfed.org/fred"
```

```
# Series IDs verified directly against FRED's own documentation/pages.
```

```
# None = not hardcoded, will use search fallback at runtime.
```

```
VERIFIED_SERIES = {
```

```
    "USD": {
```

```
        "InterestRate": "FEDFUNDS",
```

```
        # Federal funds effective rate, leve
```

```

        "CPI_YoY": "CPIAUCSL",          # CPI index -> we compute YoY %% our
        "GDP_QoQ": "GDP",              # Nominal GDP index -> compute QoQ %
        "Unemployment": "UNRATE",      # already a rate
        "TradeBalance_Bn": "BOPGSTB",  # Trade balance, goods & services, $
        "RetailSales_MoM": "RSAFS",    # Retail sales index -> compute MoM
    },
    "EUR": {
        "InterestRate": "ECBDFR",      # ECB deposit facility rate, already
        "CPI_YoY": "CPHPTT01EZM659N",  # HICP, already YoY growth rate
        "GDP_QoQ": None,
        "Unemployment": None,
        "TradeBalance_Bn": None,
        "RetailSales_MoM": None,
    },
    "GBP": {
        "InterestRate": "BOERUKM",      # Bank of England policy rate
    },
    "JPY": {
        "InterestRate": "IRSTCB01JPM156N", # BoJ central bank rate (OECD MEI)
    },
    "CAD": {
        "InterestRate": "IRSTCB01CAM156N", # BoC central bank rate (OECD MEI)
    },
}

# Series that are LEVELS (index or $) needing us to compute a growth rate,
# vs series that are already expressed as a rate/percent-change.
NEEDS_YOY_CALC = {"CPIAUCSL"}
NEEDS_QOQ_CALC = {"GDP"}
NEEDS_MOM_CALC = {"RSAFS"}
IS_MILLIONS_USD = {"BOPGSTB"} # convert to billions

# Search fallback query text per metric -- kept generic so it applies
# across countries; country name/currency is appended at search time.
SEARCH_QUERY_TEMPLATES = {
    "InterestRate": "{country} central bank policy interest rate",
    "CPI_YoY": "{country} consumer price index all items growth rate same period",
    "GDP_QoQ": "{country} real gross domestic product growth rate",
    "Unemployment": "{country} harmonized unemployment rate",
    "TradeBalance_Bn": "{country} trade balance goods and services",
    "RetailSales_MoM": "{country} retail trade volume growth rate",
}

CURRENCY_TO_COUNTRY = {
    "USD": "United States", "EUR": "Euro Area", "GBP": "United Kingdom",
    "JPY": "Japan", "AUD": "Australia", "CAD": "Canada", "CHF": "Switzerland",
    "NZD": "New Zealand", "MXN": "Mexico", "ZAR": "South Africa",

```

```

    "TRY": "Turkey", "SEK": "Sweden",
}

TEMPLATE_COLUMNS = ["InterestRate", "RateBias", "CPI_YoY", "GDP_QoQ",
                    "Unemployment", "PMI_Manufacturing", "PMI_Services",
                    "TradeBalance_Bn", "RetailSales_MoM"]

def _get(url, params, retries=3):
    for attempt in range(retries):
        r = requests.get(url, params=params, timeout=15)
        if r.status_code == 200:
            return r.json()
        if r.status_code == 429: # rate limited
            time.sleep(2 * (attempt + 1))
            continue
        r.raise_for_status()
    raise RuntimeError(f"FRED request failed after {retries} retries: {url}")

def search_series_id(api_key: str, query: str) -> str | None:
    """Best-effort: returns the top-ranked FRED series ID for a search query, or
    try:
        data = _get(f"{FRED_BASE}/series/search", {
            "search_text": query, "api_key": api_key, "file_type": "json",
            "order_by": "search_rank", "limit": 1,
        })
        seriess = data.get("series", [])
        return seriess[0]["id"] if seriess else None
    except Exception as e:
        print(f"[warn] FRED search failed for '{query}': {e}", file=sys.stderr)
        return None

def fetch_observations(api_key: str, series_id: str, n: int = 24) -> pd.Series:
    """Fetches the most recent n observations for a series, newest last."""
    data = _get(f"{FRED_BASE}/series/observations", {
        "series_id": series_id, "api_key": api_key, "file_type": "json",
        "sort_order": "desc", "limit": n,
    })
    obs = data.get("observations", [])
    vals, dates = [], []
    for o in obs:
        if o["value"] not in (".", "", None):
            vals.append(float(o["value"]))
            dates.append(o["date"])
    if not vals:

```

```

        raise ValueError(f"No usable observations for series {series_id}")
    s = pd.Series(vals[::-1], index=pd.to_datetime(dates[::-1])) # oldest first
    return s

def resolve_metric_value(api_key: str, series_id: str) -> float:
    """Fetches a series and returns the latest value, applying the right
    transformation (YoY/QoQ/MoM %% change, or unit conversion) based on
    what kind of series it is."""
    s = fetch_observations(api_key, series_id, n=30)

    if series_id in NEEDS_YOY_CALC:
        if len(s) < 13:
            raise ValueError(f"Not enough history on {series_id} for YoY calc")
        return float((s.iloc[-1] / s.iloc[-13] - 1) * 100)

    if series_id in NEEDS_QOQ_CALC:
        if len(s) < 2:
            raise ValueError(f"Not enough history on {series_id} for QoQ calc")
        return float((s.iloc[-1] / s.iloc[-2] - 1) * 100)

    if series_id in NEEDS_MOM_CALC:
        if len(s) < 2:
            raise ValueError(f"Not enough history on {series_id} for MoM calc")
        return float((s.iloc[-1] / s.iloc[-2] - 1) * 100)

    val = float(s.iloc[-1])
    if series_id in IS_MILLIONS_USD:
        val = val / 1000.0 # millions -> billions
    return val

def resolve_rate_bias(api_key: str, series_id: str) -> int:
    """Compares the latest policy rate to its value ~3 observations back
    to infer direction: 1 = hiking/hawkish, -1 = cutting/dovish, 0 = held."""
    try:
        s = fetch_observations(api_key, series_id, n=6)
        if len(s) < 2:
            return 0
        delta = s.iloc[-1] - s.iloc[0]
        if delta > 0.05:
            return 1
        elif delta < -0.05:
            return -1
        return 0
    except Exception:
        return 0

```

```

def fetch_currency_row(api_key: str, currency: str) -> dict:
    country = CURRENCY_TO_COUNTRY.get(currency, currency)
    hardcoded = VERIFIED_SERIES.get(currency, {})
    row = {}

    for metric in ["InterestRate", "CPI_YoY", "GDP_QoQ", "Unemployment",
                  "TradeBalance_Bn", "RetailSales_MoM"]:
        series_id = hardcoded.get(metric)
        if series_id is None:
            query = SEARCH_QUERY_TEMPLATES[metric].format(country=country)
            series_id = search_series_id(api_key, query)

        if series_id is None:
            print(f"[warn] {currency}/{metric}: no series found, leaving blank",
                  row[metric] = np.nan
                  continue

        try:
            row[metric] = resolve_metric_value(api_key, series_id)
        except Exception as e:
            print(f"[warn] {currency}/{metric} ({series_id}): fetch failed ({e}),
                  row[metric] = np.nan

    # RateBias: derive from the interest rate series' recent trend
    rate_series_id = hardcoded.get("InterestRate") or search_series_id(
        api_key, SEARCH_QUERY_TEMPLATES["InterestRate"].format(country=country))
    row["RateBias"] = resolve_rate_bias(api_key, rate_series_id) if rate_series_i

    # PMI: no reliable free source -- left blank for optional manual entry
    row["PMI_Manufacturing"] = np.nan
    row["PMI_Services"] = np.nan

    return row

def build_fundamental_dataframe(api_key: str, currencies: list, pause: float = 0.
    """Builds the full fundamental data table by querying FRED for each currency.
    `pause` adds a small delay between currencies to stay well under FRED's rate
    rows = {}
    for cur in currencies:
        print(f"Fetching {cur} ({CURRENCY_TO_COUNTRY.get(cur, cur)})...", file=sys
        rows[cur] = fetch_currency_row(api_key, cur)
        time.sleep(pause)

    df = pd.DataFrame.from_dict(rows, orient="index")

```

```

df.index.name = "Currency"
return df[TEMPLATE_COLUMNS]

if __name__ == "__main__":
    import argparse
    import os

    parser = argparse.ArgumentParser(description="Fetch fundamental data from FRED")
    parser.add_argument("--fred-key", default=os.environ.get("FRED_API_KEY"),
                        help="FRED API key (or set FRED_API_KEY env var)")
    parser.add_argument("--currencies", default="USD,EUR,GBP,JPY,AUD,CAD,CHF,NZD",
                        help="Currencies to fetch")
    parser.add_argument("--out", default="fundamental_data_auto.csv")
    args = parser.parse_args()

    if not args.fred_key:
        print("Error: no FRED API key. Pass --fred-key or set FRED_API_KEY.\n")
        print("Get a free key at https://fredapi.stlouisfed.org/docs/api/api_key.")
        sys.exit(1)

    currencies = [c.strip().upper() for c in args.currencies.split(",")]
    df = build_fundamental_dataframe(args.fred_key, currencies)
    df.to_csv(args.out)
    print(f"\nWrote {args.out}:")
    print(df.to_string(float_format=lambda x: f"{x:.2f}" if pd.notna(x) else "NaN"))

```